

Intro2SE - Testing - 6

SmartATS



Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS

Table of Contents

Objectives.....	1
1 Member Contribution Assessment.....	2
2 Test plan	5
2. 1. Scope and Objectives	<i>Error! Bookmark not defined.</i>
2. 2. Testing Approaches and Techniques	<i>Error! Bookmark not defined.</i>
3 Test cases	7
3.1 List of test cases.....	7
3.1.1 Profile Ingestion & Telemetry	7
3.1.2 AI Parsing & Semantic Ranking	9
3.1.3 Cross-Channel Profile Enrichment.....	10
3.1.4 ABAC Security & Interview Scheduling.....	12
3.2 Test case specifications	14
3.2.1 Profile Ingestion Pipeline & Global UI Workspace	14
3.2.2 AI Resume Parsing & Semantic Ranking	15
3.2.3 Cross-Channel Profile Enrichment.....	25
3.2.4 ABAC Security & Scheduling.....	31
4 AI Usage Declaration	38
5 Presentation	39
6 Reflective Report	40

Software Testing

Objectives

This document focus on the following topics:

- ✓ Completing the Software Testing document with the following sections:
 - Test Plan
 - Test Cases
- ✓ Understanding the Software Testing document.
- ✓ This document will be used as input for AI tools to verify the quality of subsequent project artifacts.

All project artifacts must remain consistent and synchronized.

For example, if the project proposal is modified during this phase, a new version of the proposal must be documented.

By the conclusion of the project, all versions of every artifact must be submitted to demonstrate the evolution of your work.

1 Member Contribution Assessment

24127254 - Hồ Đình Trí - 25%

Task Evidence:

- **Task SAT-TSK-01: Feature Ownership & Quality Assurance for Flow A (Ingestion & WebSockets)**
 - **Task Details:**
 - Set up the initial document skeleton for the testing report in alignment with Template3-Testing.docx.
 - Managed presentation video aggregation and YouTube upload (Section 5).
 - Developed and executed unit and integration test suites for the Ingestion Gateway (FastAPI: IngestionRouter).
 - Verified binary PDF Magic Byte headers (%PDF-) and enforced file size limit boundaries (10MB). Validated real-time WebSocket connection lifecycle (WebSocketRouter, WebSocketConnectionManager) and progress telemetry streaming (PARSING_STARTED, ANALYSIS_STARTED, AI_ANALYTICS_COMPLETE).
 - Authored and documented all associated test cases in Sections 3.1 and 3.2.
 - **Visual Evidence:**

✓ AP-56 Ingestion & Telemetry	Đinh Trí	Đinh Trí	= Medium	Done ✓
AP-74 Tổng hợp Link Video Presentation YouTube...	Đinh Trí	Đinh Trí	= Medium	Done ✓
AP-60 Viết Unit Test & Integration Test cho Ingest...	Đinh Trí	Đinh Trí	= Medium	Done ✓
AP-61 Soạn thảo Test Cases(Mục 3.1 và 3.2)	Đinh Trí	Đinh Trí	= Medium	Done ✓
AP-62 Tổng hợp Task Evidence (Jira/Git) của bản ...	Đinh Trí	Đinh Trí	= Medium	Done ✓

24127382 - Trần Nhật Hoàng - 25%

Task Evidence:

- **Task SAT-TSK-02: Feature Ownership & Quality Assurance for Flow B (AI Resume Parsing & Ranking)**

- **Task Details:**

- Authored the Reflective Report section analyzing template utility and area for improvement.
- Developed and executed test scripts targeting the AI Orchestration Engine (AIOrchestrationLoop), Model Context Protocol host (MCPHost), and Skill Registry.
- Conducted precision testing on semantic ranking and pgvector 1536-dimensional cosine similarity search against candidate embeddings in PostgreSQL.
- Verified contextual prompt handling and AI Chat assistant output structured formats.
- Authored and documented all associated test cases in Sections 3.1 and 3.2.

- **Visual Evidence:**

✓ AP-57 AI Parsing & Ranking	Hoàng Tran	Đinh Trí	= Medium	Done ✓
AP-66 Tổng hợp Task Evidence (Jira/Git) của bản ...	Hoàng Tran	Đinh Trí	= Medium	Done ✓
AP-65 Viết phần Reflective Report (Mục 6 trong R...	Hoàng Tran	Đinh Trí	= Medium	Done ✓
AP-64 Soạn thảo Test Cases(Mục 3.1 & 3.2 trong ...	Hoàng Tran	Đinh Trí	= Medium	Done ✓
AP-63 Viết Test Suite cho pgvector Cosine Similar...	Hoàng Tran	Đinh Trí	= Medium	Done ✓

24127252 - Nguyễn Khánh Toàn - 25%

Task Evidence:

- **Task SAT-TSK-03: Feature Ownership & Quality Assurance for Flow C (Cross-Channel Enrichment)**

- **Task Details:**

- Authored the AI Usage Declaration section following project guidelines.
- Formulated and executed test suites for background crawling engines (GitHubScraper, LinkedInScraperService), including API rate-limiting resilience and fallback mechanisms.
- Verified data consistency and conflict resolution of the idempotent data merge strategy (idempotent_merge) updating PostgreSQL JSONB structures (UPSERT ON CONFLICT).
- Validated enrichment analytics logic (Radar skill matrices and confidence score calculations). Authored and documented all associated test cases in Sections 3.1 and 3.2.

- **Visual Evidence:**

✓ AP-58 Enrichment & Merging	Toàn Nguyễn Khánh	Đinh Trí	= Medium	Done ✓
AP-67 Viết Test Suite cho Scrapers (GitHub/Linke...	Toàn Nguyễn Khánh	Đinh Trí	= Medium	Done ✓
AP-68 Soạn thảo Test Cases (Mục 3.1 & 3.2 trong ...	Toàn Nguyễn Khánh	Đinh Trí	= Medium	Done ✓
AP-69 Viết phần AI Usage Declaration (Mục 4 tro...	Toàn Nguyễn Khánh	Đinh Trí	= Medium	Done ✓
AP-70 Tổng hợp Task Evidence (Jira/Git) của bản ...	Toàn Nguyễn Khánh	Đinh Trí	= Medium	Done ✓

24127337 - Nguyễn Tiến Cường - 25%

Task Evidence:

- **Task SAT-TSK-04: Feature Ownership & Quality Assurance for Flow D (ABAC Security & Scheduling)**

- **Task Details:**

- Authored the overall Test Plan & Strategy section (Section 2).

- Designed and executed security testing for the ABAC Data Masking Middleware (ABACEngine), verifying dynamic PII redaction (full_name, email, phone, salary_expectation for tech_lead role vs full clearance for HR/Admin).
- Tested the Sweep-Line calendar overlap scheduling algorithm and Google Calendar v3 FreeBusy API integrations (SchedulingService, GoogleCalendarService, EmailNotifier).
- Authored and documented all associated test cases in Sections 3.1 and 3.2.

○ **Visual Evidence:**

✓ AP-59 Security & Scheduling	CT Cường Nguyễn Tiến	Đinh Trí	= Medium	Done ✓
AP-73 Soạn thảo Test Cases (Mục 3.1 & 3.2 trong ...)	CT Cường Nguyễn Tiến	Đinh Trí	= Medium	Done ✓
AP-72 Viết Test Suite cho ABACEngine Masking M...	CT Cường Nguyễn Tiến	Đinh Trí	= Medium	Done ✓
AP-71 Viết Test Strategy & Test Environment (Mục ...)	CT Cường Nguyễn Tiến	Đinh Trí	= Medium	Done ✓
AP-75 Tổng hợp Task Evidence của bản thân vào ...	CT Cường Nguyễn Tiến	Đinh Trí	= Medium	Done ✓

2 Test plan

Written by: Nguyễn Tiến Cường (24127337)

Edited by: Hồ Đình Trí (24127254)

Reviewed by: Trần Nhật Hoàng (24127382), Nguyễn Khánh Toàn (24127252)

2. 1. Scope and Objectives

The software testing scope for the Applicant Tracking System (SmartATS) covers 4 core operational flows, administrative gateways, and backend microservice components to ensure high quality, privacy compliance, and algorithmic precision:

- **Flow A (Profile Ingestion & Telemetry):** Validating FastAPI binary upload gateways, PDF Magic Byte signatures (%PDF-), strict 10MB payload size boundaries, Azure Blob Storage

signed SAS URLs, and real-time bidirectional WebSocket telemetry streaming (PARSING_STARTED, ANALYSIS_STARTED, ENRICHED).

- **Flow B (AI Resume Parsing & Semantic Ranking):** Validating AI Orchestration Engine (AIOrchestrationLoop), Model Context Protocol (MCP) host tool routing, structured schema extraction, and 1536-dimensional pgvector cosine similarity ranking with mandatory hard-skill filtering.
- **Flow C (Cross-Channel Profile Enrichment):** Validating OSINT background scrapers (GitHub REST API, LinkedIn crawler via Apify), rate-limiting resilience, idempotent conflict resolution (idempotent_merge), PostgreSQL JSONB UPSERT persistence, and 5-dimensional Radar skill matrix calculations.
- **Flow D (ABAC Security & Interview Scheduling):** Validating the Attribute-Based Access Control (ABACEngine) dynamic default-deny data masking middleware, role-based clearance (HR/Admin vs Tech Lead), Sweep-Line overlapping interval scheduling algorithm ($O(N \log N)$), Google Calendar v3 FreeBusy synchronization with automatic OAuth token refresh resilience, and multi-channel notification dispatch (Email & Slack) formatted in local GMT+7 timezone.
- **Administrative & Job Posting Lifecycle:** Validating RBAC middleware access enforcement (HTTP 403 Forbidden), admin role mutations, account suspension guards (is_approved), and job posting lifecycle status/expiration guards.

Objectives & Quality Criteria: (1) Verify 100% functional compliance of candidate ingestion, parsing, enrichment, and scheduling workflows; (2) Guarantee zero-leakage of Candidate Personally Identifiable Information (PII) to technical interviewers in strict accordance with GDPR data minimization standards; (3) Verify algorithmic precision of the Sweep-Line calendar overlap algorithm across normal, boundary, and disjoint availability conditions; (4) Attain a 100% test pass rate across all automated unit and integration test suites.

2. 2. Testing Approaches and Techniques

The testing strategy employs a multi-tiered verification hierarchy combining deterministic unit testing, integrated API testing, and specialized security verification:

- **Unit Testing:** Implemented using pytest and pytest-asyncio to verify isolated components without active database dependencies (e.g., SweepLineService, ABAC data masking functions, Pydantic schema validation).
- **Integration Testing:** Executed via fastapi.testclient.TestClient to test end-to-end API routes, authentication dependency injection (get_current_user overrides), and role authorization matrix enforcement.
- **Security & Privacy Testing (ABAC Masking):** Automated assertions verifying dynamic PII redaction across all nested JSON structures, default-deny behavior on unwhitelisted schema fields, and preservation of non-string data types (0, False, []) to prevent serialization failures.
- **Equivalence Partitioning (EP) & Boundary Value Analysis (BVA):** Systematic partitioning of input domains and state spaces into mutually exclusive equivalence classes (valid clearance, restricted role, default-deny boundary, valid overlap $\geq 45m/60m$, sub-minimum discard $< 45m$, and zero overlap disjoint sets) to ensure exhaustive test coverage without redundant test cases.
- **External Dependency Mocking:** Virtualization of third-party external dependencies (Google Calendar v3 FreeBusy API, Google OAuth refresh service, Apify LinkedIn crawler, Slack webhooks, and Azure Blob storage) using unittest.mock (AsyncMock, MagicMock) for deterministic and fast test execution.

3 Test cases

3.1 List of test cases

Provide the name of each test case, the associated feature, and a brief description (if applicable).

Teams are required to test all possible scenarios for each feature.

Test cases should be prioritized, focusing specifically on the 5 most critical features.

3.1.1 Profile Ingestion & Telemetry

Written by: Hồ Đình Trí (24127254)

Edited by: Trần Nhật Hoàng (24127382)

Reviewed by: Nguyễn Tiến Cường (24127337)

Seq	Test Case	Feature	Description
1	PDF Magic Byte Verification	Ingestion Gateway	Rejects non-PDF file disguised with .pdf extension
2	File Size Boundary Capping	Ingestion Gateway	Rejects file upload exceeding 10MB limit (10,485,760 bytes)
3	MIME Type Enforcement	Ingestion Gateway	Validates application/pdf Content-Type header integrity
4	CV Signed SAS URL Generation	Ingestion Gateway	Grants temporary SAS storage access with HTTP 302/307 redirect
5	WS Handshake & Registration	Telemetry WebSockets	Establishes WebSocket connection and registers socket session
6	Cached Enriched Profile Delivery	Telemetry WebSockets	Instantly dispatches profile payload when cached data exists
7	Live Broadcast State Telemetry	Telemetry WebSockets	Broadcasts ENRICHED and ENRICHMENT_FAILED live status frames
8	WS Disconnect Resource Cleanup	Telemetry WebSockets	Automatically purges socket memory allocations on client disconnect

3.1.2 Job Posting Validation

Written by: Hồ Đình Trí (24127254)

Edited by: Trần Nhật Hoàng (24127382)

Reviewed by: Nguyễn Tiến Cường (24127337)

Seq	Test case	Feature	Description
1	Job Status State Guard	Job Posting Validation	Enforces status transitions (DRAFT, PUBLISHED, CLOSED, ARCHIVED)
2	Job Posting Expiration Guard	Job Posting Validation	Rejects candidate applications past the expires_at timestamp

3.1.3 Admin Management

Written by: Hồ Đình Trí (24127254)

Edited by: Trần Nhật Hoàng (24127382)

Reviewed by: Nguyễn Tiến Cường (24127337)

Seq	Test case	Feature	Description
1	Admin RBAC Access Enforcement	Admin Management	Rejects non-admin users with HTTP 403 Forbidden status
2	Admin User & Role Mutation	Admin Management	Updates user profile details, is_approved status, and roles
3	Admin Update Payload Validation	Admin Management	Validates strict structure of user role update payloads

3.1.4 AI Parsing & Semantic Ranking

Written by: Trần Nhật Hoàng (24127382)

Edited by: Nguyễn Khánh Toàn (24127252)

Reviewed by: Nguyễn Tiến Cường (24127337)

Seq	Test Case	Feature	Description
1	Successful Resume	AI Resume	Successfully extracts structured candidate

	Parsing	Parsing & Semantic Ranking	information such as skills, education, experience, and summary from a valid resume.
2	Invalid or Unprocessable Resume	AI Resume Parsing & Semantic Ranking	Rejects or safely handles resumes that are corrupted, unsupported, empty, or cannot be parsed correctly.
3	Missing or Incomplete Resume Information	AI Resume Parsing & Semantic Ranking	Correctly handles resumes with missing fields without incorrectly generating or assuming candidate information.
4	Semantic Resume-JD Matching	AI Resume Parsing & Semantic Ranking	Ranks candidates based on semantic relevance between resume content and job description, beyond exact keyword matching.
5	Hard Skill Filtering and Semantic Ranking	AI Resume Parsing & Semantic Ranking	Filters candidates based on required skills and ranks the remaining candidates according to semantic relevance to the job requirements.

3.1.5 Cross-Channel Profile Enrichment

Written by: Nguyễn Khánh Toàn (24127252)

Edited by: Hồ Đình Trí (24127254)

Reviewed by: Nguyễn Tiến Cường (24127337)

Seq	Test case	Feature	Description
1	GitHub Scraper — Successful Profile Fetch	GitHub Crawler	Verifies that a valid GitHub username triggers a successful API call, resolves the top-3 programming languages by repository size, and fetches README

			content from up to 5 most recently updated repositories
2	LinkedIn Scraper — Error & Fallback Handling	LinkedIn Crawler (Apify)	Verifies that the scraper correctly handles failure scenarios: missing API token raises an immediate error, and an empty Apify dataset causes the worker to gracefully mark the candidate as enrichment-failed without crashing
3	Idempotent Merge — Deduplication & Score Maximum	idempotent_merge Conflict Resolution	Validates that re-enriching a candidate merges semantic tags as a deduplicated union (capped at 8), takes the element-wise maximum across all 5 radar skill dimensions, and resolves duplicate LinkedIn company entries by keeping the newest record
4	UPSERT Persistence — GitHub & LinkedIn JSONB Storage	Supabase UPSERT (github_profiles / linkedin_profiles)	Confirms that scraped profile data is correctly inserted on first run and overwritten (not duplicated) on re-enrichment, with nested JSONB structures (language maps,

			experience/education arrays with date objects) stored intact
5	Radar Analytics — Skill Score & Confidence Calculation	generate_analytics + analyze_github_local_fallback	Validates the radar scoring formula (base 25 + keyword hits × 12 + language bonus), the confidence score formula (capped at 99.0), and the pre-enrichment scaling factor (× 0.75) against controlled inputs
6	Enrichment Worker — End-to-End Pipeline	enrichment_worker (8-step pipeline)	Executes the full enrichment workflow for a candidate with both social profiles: candidate persisted → social links updated → GitHub & LinkedIn scraped → data stored via UPSERT → analytics generated → status set to ENRICHED → WebSocket notification delivered

3.1.6 ABAC Security & Interview Scheduling

Written by: Nguyễn Tiến Cường (24127337)

Edited by: Nguyễn Khánh Toàn (24127252)

Reviewed by: Hồ Đình Trí (24127254)

Seq	Test Case	Feature	Description
1	ABAC Full Access Clearance	ABAC Security Engine	Verifies that HR and Admin roles receive 100% unredacted candidate profiles, retaining all PII and sensitive compensation data.
2	ABAC Tech Role PII Masking	ABAC Security Engine	Verifies that Tech Lead role has all candidate PII (name, email, phone, salary, avatar) redacted to "***", while technical skills and repositories remain visible.
3	ABAC Default-Deny & Type Preservation	ABAC Security Engine	Enforces default-deny on unwhitelisted schema fields and ensures non-string types (0, False, []) are preserved to prevent Pydantic 500 serialization errors.
4	Sweep-Line Multi-Interviewer Overlap	Sweep-Line Scheduling Algorithm	Validates that the Sweep-Line algorithm ($O(N \log N)$) accurately computes common free intervals across multiple interviewers, slices 45m slots, and flags $\geq 60m$ slots as "Recommended".
5	Sweep-Line Sub-Minimum Discard ($<$	Sweep-Line Scheduling Algorithm	Verifies that overlapping free windows strictly less

	45m)		than the 45-minute threshold (e.g., 30m) are safely discarded.
6	Sweep-Line Disjoint Schedules (Zero Overlap)	Sweep-Line Scheduling Algorithm	Confirms that completely conflicting interviewer schedules return an empty slot list ([]) gracefully without errors or infinite loops.
7	Google Calendar Token Refresh Resilience	Google Calendar & OAuth Integration	Verifies that an expired Google Calendar access token triggers automatic OAuth token refresh via <code>calendar_refresh_token</code> and completes the FreeBusy query.
8	Interview Booking & Multi-Channel Notification	Interview Booking & Notifiers	Confirms booking state mutation (CONFIRMED), GMT+7 timestamp formatting, Google Meet URL creation, and simultaneous Email and Slack notification dispatch.

3.2 Test case specifications

3.2.1 Profile Ingestion Pipeline & Global UI Workspace

Written by: Hồ Đình Trí (24127254)

Edited by: Trần Nhật Hoàng (24127382)

Reviewed by: Nguyễn Tiến Cường (24127337)

Test Case 1: PDF Magic Byte Verification

	<i>PDF Magic Byte Verification</i>
Related feature	IngestionRouter Magic Byte Check
Context	User attempts to upload a script/executable file renamed to resume.pdf
Input Data	File spoofed_cv.pdf (Header: MZ..., File Size: 1.5MB)
Expected Output	HTTP Status Code 400 Bad Request, Payload: {"detail": "Invalid file format: Magic bytes do not match %PDF-"} }
Test steps	1. Send POST request to /api/v1/ingestion/upload with spoofed_cv.pdf. 2. Inspect HTTP status code and response payload JSON. 3. Verify async pipeline task queue was NOT triggered.
Actual Output	HTTP 400 Bad Request returned with detail message. Background worker task was not created.
Result	Passed

Test Case 2: File Size Boundary Capping

<i>Test case</i>	<i>File Size Boundary Capping</i>
Related feature	IngestionRouter Boundary Enforcement ($\leq 10,485,760$ bytes)
Context	User uploads an oversized PDF resume exceeding the maximum 10MB limit
Input Data	File large_portfolio.pdf (Size: 10,485,761 bytes / ~ 10.01 MB)
Expected Output	HTTP Status Code 413 Payload Too Large, Payload: {"detail": "File size exceeds limit of 10MB"} }
Test steps	1. Send POST request to /api/v1/ingestion/upload with 10.01MB file. 2. Assert HTTP response status code is 413. 3. Verify upload stream to storage was aborted.
Actual Output	HTTP 413 Payload Too Large received. Azure Blob Storage upload bypassed.
Result	Passed

Test case 3: MIME Type Enforcement

<i>Test case</i>	<i>MIME Type Enforcement</i>
Related feature	IngestionRouter Content-Type Validation
Context	Client submits a file upload with an unsupported Content-Type header
Input Data	File document.docx, Header Content-Type: application/msword
Expected Output	HTTP Status Code 415 Unsupported Media Type
Test steps	1. Send POST request with non-PDF MIME type. 2. Verify system rejects upload prior to processing byte payload.
Actual Output	HTTP 415 Unsupported Media Type returned as expected.
Result	Passed

Test Case 4: CV Signed SAS URL Generation

<i>Test case</i>	<i>CV Signed SAS URL Generation</i>
Related feature	Azure Blob Storage Integration & Signed URL Grant
Context	Authenticated user requests direct secure view of candidate CV file
Input Data	GET /api/v1/ingestion/cv/{candidate_uuid}/download
Expected Output	HTTP Status Code 302 Found / 307 Temporary Redirect with valid Azure Blob SAS URL token
Test steps	1. Issue GET request with valid Authorization header. 2. Inspect HTTP status code and Location redirect header. 3. Verify SAS URL parameters (signature, expiration duration).
Actual Output	HTTP 307 Redirect issued with secure SAS URL token attached in Location header.
Result	Passed

Test Case 5: WS Handshake & Registration

<i>Test case</i>	<i>WS Handshake & Registration</i>
Related feature	WebSocketConnectionManager Session Lifecycle
Context	Client UI opens real-time telemetry socket for active job_id
Input Data	Connection Request: /ws/v1/analysis/{uuid}
Expected Output	WebSocket Handshake Accepted (HTTP 101 Switching Protocols), socket registered in ConnectionManager
Test steps	1. Initiate WS client connection to /ws/v1/analysis/{uuid}. 2. Assert connection handshake success. 3. Check internal connection registry for active socket instance.
Actual Output	WS Handshake established successfully. Active connection logged in manager store.
Result	Passed

Test Case 6: Cached Enriched Profile Delivery

<i>Test case</i>	<i>Cached Enriched Profile Delivery</i>
Related feature	WebSocketRouter Instant Cache Dispatch
Context	Client connects to WS endpoint for a candidate whose analysis is already complete
Input Data	WS Connect to /ws/v1/analysis/{uuid} (Target candidate pre-analyzed in PostgreSQL)
Expected Output	Immediate single WS frame transmission containing cached enriched profile data
Test steps	1. Pre-seed DB with completed profile data. 2. Connect WS socket. 3. Receive initial WS message frame. 4. Assert payload matches cached DB record immediately.
Actual Output	Received full enriched JSON payload within 15ms upon connection opening.
Result	Passed

Test Case 7: Live Broadcast State Telemetry

<i>Test case</i>	<i>Live Broadcast State Telemetry</i>
Related feature	Real-Time Telemetry Streaming (ENRICHED / ENRICHMENT_FAILED)
Context	Async pipeline state changes during live UI session
Input Data	Pipeline worker event trigger (ENRICHED or ENRICHMENT_FAILED)
Expected Output	Active client receives real-time JSON frame with corresponding event state
Test steps	1. Maintain active WS session. 2. Trigger background worker completion event. 3. Intercept incoming WS frame. 4. Assert status field equals ENRICHED or ENRICHMENT_FAILED.
Actual Output	Received frame {"status": "ENRICHED", "data": {...}} upon task completion.
Result	Passed

Test Case 8: WS Disconnect Resource Cleanup

<i>Test case</i>	<i>WS Disconnect Resource Cleanup</i>
Related feature	WebSocketConnectionManager Memory Management
Context	Client unexpectedly drops network connection or closes browser tab
Input Data	Explicit Client Connection Close / Socket Disconnect signal
Expected Output	Socket instance removed from memory pool; no lingering zombie connections
Test steps	1. Open WS connection. 2. Force disconnect client socket. 3. Query WebSocketConnectionManager.active_connections count.

	4. Assert connection count decrements to zero.
Actual Output	Active connection list decremented immediately upon disconnect event.
Result	Passed

3.2.2 Admin Management

Written by: Hồ Đình Trí (24127254)

Edited by: Trần Nhật Hoàng (24127382)

Reviewed by: Nguyễn Tiến Cường (24127337)

Test Case 1: Admin RBAC Access Enforcement

<i>Test case</i>	<i>Admin RBAC Access Enforcement</i>
Related feature	Global RBAC Middleware (test_admin_router.py)
Context	User with non-admin role (candidate or tech_lead) attempts to access admin endpoint
Input Data	GET /api/v1/admin/users, Bearer JWT Token (role: tech_lead)
Expected Output	HTTP Status Code 403 Forbidden, Payload: {"detail": "Insufficient permissions"}
Test steps	1. Issue request to /api/v1/admin/users with non-admin JWT. 2. Assert HTTP response status code is 403.
Actual Output	HTTP 403 Forbidden returned. Access denied.
Result	Passed

Test Case 2: Admin User & Role Mutation

<i>Test case</i>	<i>Admin User & Role Mutation</i>
Related feature	Admin User Management (test_admin_router.py)
Context	System Admin updates user clearance (is_approved) and assigns new role
Input Data	PATCH /api/v1/admin/users/{uuid}, Payload: {"is_approved": true, "role": "hr_manager"}
Expected Output	HTTP Status Code 200 OK, response confirms updated user

	entity in DB
Test steps	1. Send PATCH request with Admin JWT token. 2. Verify response status code 200. 3. Inspect PostgreSQL users table to verify is_approved=true and role='hr_manager'.
Actual Output	HTTP 200 OK returned. User role updated in PostgreSQL.
Result	Passed

Test Case 3: Admin Update Payload Validation

Test case	<i>Admin Update Payload Validation</i>
Related feature	FastAPI Schema Payload Validation (test_admin_router.py)
Context	Admin sends malformed role string or invalid JSON structure
Input Data	PATCH /api/v1/admin/users/{uuid}, Payload: {"role": "invalid_super_role"}
Expected Output	HTTP Status Code 422 Unprocessable Entity
Test steps	1. Send request with invalid Enum role value. 2. Verify Pydantic validation interceptor catches error.
Actual Output	HTTP 422 Unprocessable Entity returned with schema validation details.
Result	Passed

3.2.3 Job Posting Validation

Written by: Hồ Đình Trí (24127254)

Edited by: Trần Nhật Hoàng (24127382)

Reviewed by: Nguyễn Tiến Cường (24127337)

Test Case 1: Job Status State Guard

Test case	<i>Job Status State Guard</i>
Related feature	Job Posting Lifecycle Guard (test_job_posting_validation.py)
Context	Candidate attempts to apply for a job posting in DRAFT,

	CLOSED, or ARCHIVED state
Input Data	POST /api/v1/jobs/{job_id}/apply for job with status: CLOSED
Expected Output	HTTP Status Code 400 Bad Request, Payload: {"detail": "Job posting is not active"}
Test steps	1. Set job posting state to CLOSED. 2. Submit application request. 3. Assert HTTP response status is 400. Repeat for DRAFT and ARCHIVED.
Actual Output	HTTP 400 Bad Request returned across all non-PUBLISHED job states.
Result	Passed

Test Case 2: Job Posting Expiration Guard

<i>Test case</i>	<i>Job Posting Expiration Guard</i>
Related feature	Job Posting Expiration Logic (test_job_posting_validation.py)
Context	Candidate attempts to apply after the expires_at deadline has passed
Input Data	Application request for job with expires_at: 2026-01-01T00:00:00Z
Expected Output	HTTP Status Code 400 Bad Request, Payload: {"detail": "Job posting has expired"}
Test steps	1. Mock current time past expires_at timestamp. 2. Issue application request. 3. Verify expiration logic blocks candidate submission.
Actual Output	HTTP 400 Bad Request returned due to expired deadline timestamp.
Result	Passed

3.2.4 AI Resume Parsing & Semantic Ranking

Written by: Trần Nhật Hoàng (24127382)

Edited by: Nguyễn Khánh Toàn (24127252)

Reviewed by: Nguyễn Tiến Cường (24127337)

Test Case 1: Successful Resume Parsing

<i>Test Case</i>	<i>Successful Resume Parsing</i>
Related feature	AI Resume Parsing
Context	The system receives a valid and readable candidate resume and processes it through the resume parsing and AI enrichment pipeline.
Input Data	A valid PDF resume containing candidate information such as personal information, skills, education, work experience, and summary.
Expected Output	The system successfully extracts and structures the relevant candidate information. The parsed data is stored correctly and the resume is available for further candidate search and ranking.
Test steps	1. Upload a valid resume. 2. Trigger the resume parsing pipeline. 3. Parse the resume content. 4. Run AI enrichment to extract structured information. 5. Verify the stored candidate data.
Actual Output	To be recorded during testing.
Result	Pass / Fail

Test Case 2: Resume Parsing with Missing Information

<i>Test Case</i>	<i>Resume Parsing with Missing Information</i>
Related feature	AI Resume Parsing
Context	The system processes a resume that contains incomplete candidate information, such as missing education, work experience, or skills sections.
Input Data	A valid resume with one or more missing or incomplete information sections.
Expected Output	The system successfully parses the available information. Missing fields are stored as empty or null values, and the AI does not incorrectly generate or assume information that is not present in the resume.

Test steps	1. Upload a resume with incomplete information. 2. Trigger the resume parsing pipeline. 3. Run AI enrichment. 4. Verify the extracted information. 5. Check the fields corresponding to missing information.
Actual Output	To be recorded during testing.
Result	Pass / Fail

Test Case 3: Resume Parsing Failure Handling

Test Case	<i>Resume Parsing Failure Handling</i>
Related feature	AI Resume Parsing
Context	The system receives a resume that cannot be successfully processed by the parsing or AI enrichment pipeline.
Input Data	A corrupted, unreadable, unsupported, or otherwise unprocessable resume file.
Expected Output	The system handles the failure gracefully without crashing or creating incorrect candidate information. The original resume remains available, and the candidate's enrichment status is marked as failed or unavailable for further processing.
Test steps	Upload an invalid or unprocessable resume. Trigger the resume parsing pipeline. Observe the parsing or enrichment process. Verify the system's error handling behavior. Check the candidate's enrichment status and stored resume data.
Actual Output	To be recorded during testing.
Result	Pass / Fail

Test Case 4: Semantic Resume-JD Matching

Test Case	<i>Semantic Resume-JD Matching</i>
Related	Semantic Ranking

feature	
Context	The system needs to identify and rank candidates according to the semantic relevance between their resume content and a given job description.
Input Data	A job description and a set of candidate resumes with different levels of relevant experience and skills.
Expected Output	Candidates are ranked according to their semantic relevance to the job description. Candidates whose experience and qualifications are more relevant to the job requirements receive higher rankings, even when the exact keywords do not fully match.
Test steps	Create or upload a job description. Select a set of candidate resumes. Execute the candidate search and semantic ranking process. Calculate the relevance score for each candidate. Verify the returned candidate ranking.
Actual Output	To be recorded during testing.
Result	Pass / Fail

Test Case 5: Hard Skill Filtering and Semantic Ranking

Test Case	<i>Hard Skill Filtering and Semantic Ranking</i>
Related feature	Semantic Ranking
Context	The system must first filter candidates based on mandatory skills and then rank the remaining candidates according to their semantic relevance to the job description.
Input Data	A job description requiring mandatory skills such as React and TypeScript, along with a set of candidate resumes containing different combinations of required skills and relevant experience.
Expected Output	Candidates who do not possess the mandatory skills are excluded from the result. Candidates who satisfy the mandatory skill requirements are ranked based on semantic relevance to the job description, and the top-K candidates are returned.
Test steps	Create or upload a job description with mandatory skills.

	Submit the candidate search request Apply hard skill filtering to eliminate candidates who do not meet mandatory requirements. Perform semantic matching on the remaining candidates. Rank candidates according to their relevance scores. Return the top-K candidates.
Actual Output	To be recorded during testing.
Result	Pass / Fail

3.2.5 Cross-Channel Profile Enrichment

Written by: Nguyễn Khánh Toàn (24127252)

Edited by: Hồ Đình Trí (24127254)

Reviewed by: Nguyễn Tiến Cường (24127337)

Test Case 1: GitHub Scraper — Successful Profile Fetch

<i>Test case</i>	GitHub Scraper — Successful Profile Fetch
Related Feature	GitHub Crawler — fetch_github_profile
Context	A candidate has a valid github_username registered in the system. The GitHub REST API is reachable and returns a well-formed response. The enrichment worker triggers the GitHub scraping step.
Input Data	github_username = "testdev", valid GitHub API token configured, GitHub API mock returns 10 repositories with varied programming languages and sizes
Expected Output	A GitHubProfile object is constructed containing: public_repos_count = 10, a top_languages dictionary with at most 3 entries each expressed as a percentage share, and a non-empty readme_content string aggregated from the top 5 repositories (max 3,000 characters per README)
Test Steps	1. Configure a valid github_username for the candidate. 2. Trigger the enrichment pipeline for the candidate.

	3. Inspect the returned GitHub profile object. 4. Assert public_repos_count equals the number of repositories returned. 5. Assert top_languages contains at most 3 language entries. 6. Assert readme_content is non-empty and does not exceed 15,000 characters total.
Actual Output	GitHubProfile constructed successfully with public_repos_count = 10, top_languages = {"Python": 72.3, "Shell": 20.1, "Dockerfile": 7.6}, and readme_content populated from 3 repositories.
Result	Pass / Fail

Test Case 2: LinkedIn Scraper — Error & Fallback Handling

<i>Test case</i>	LinkedIn Scraper — Error & Fallback Handling
Related Feature	LinkedIn Crawler — fetch_linkedin_profile via Apify
Context	Two failure scenarios are tested: (a) the system is deployed without the Apify API token configured; (b) the Apify actor completes but the resulting dataset is empty (private or deleted LinkedIn profile).
Input Data	Scenario A: apify_api_token = None (not configured). Scenario B: linkedin_url = "https://www.linkedin.com/in/private-user/", Apify actor returns an empty items list
Expected Output	Scenario A: An error is raised immediately before any HTTP call is made, with the message indicating the missing token. Scenario B: The scraper raises a descriptive error about the empty dataset; the enrichment worker catches it, sets linkedin_profile = None, and continues without crashing. In both cases, the overall pipeline status remains recoverable.
Test Steps	1. (Scenario A) Remove APIFY_API_TOKEN from the environment. Trigger LinkedIn scraping. Assert an error is raised immediately and no network call is made. 2. (Scenario B) Restore the token. Configure the Apify mock to return an empty dataset. Trigger LinkedIn scraping. Assert the worker logs a failure, sets linkedin_profile = None, and does not

	crash. 3. Verify the overall enrichment status is set to ENRICHMENT_FAILED only if both GitHub and LinkedIn both fail.
Actual Output	Scenario A: Error raised immediately — "Missing APIFY_API_TOKEN". Scenario B: Worker catches empty-dataset error gracefully; linkedin_profile = None; pipeline continues to analytics step.
Result	Pass / Fail

Test Case 3: Idempotent Merge — Deduplication & Score Maximum

<i>Test case</i>	Idempotent Merge — Deduplication & Score Maximum
Related Feature	idempotent_merge — conflict resolution on re-enrichment
Context	A candidate has already been enriched once. A second enrichment run produces updated data with overlapping semantic tags, different skill scores, and a LinkedIn experience from the same company as before. The merge function must resolve all conflicts correctly.
Input Data	Existing profile: semantic_tags = ["#python", "#docker", "#aws", "#redis"], post_enrichment = [80, 60, 40, 25, 70], LinkedIn experience at "Google" with title "SWE I". New profile: semantic_tags = ["#python", "#react", "#typescript"], post_enrichment = [70, 75, 55, 25, 65], LinkedIn experience at "Google" with updated title "SWE II" and a new entry at "Meta"
Expected Output	Merged semantic tags: a deduplicated union of both sets, capped at 8 — "#python" appears exactly once. Merged skill scores: element-wise maximum per dimension — [80, 75, 55, 25, 70]. Merged LinkedIn experiences: "Google" entry retains the newer title "SWE II"; "Meta" entry is added; total = 2 entries (not 3).
Test Steps	1. Construct existing and new_data enriched profile objects with the inputs above. 2. Call the merge function with both objects. 3. Assert merged semantic_tags count ≤ 8 and "#python" appears

	exactly once. 4. Assert post_enrichment scores equal [80, 75, 55, 25, 70]. 5. Assert LinkedIn experiences total 2 entries, with "Google" showing title "SWE II".
Actual Output	Tags: 7 unique entries, "#python" once. Scores: [80, 75, 55, 25, 70]. Experiences: 2 entries — Google (SWE II), Meta (SWE).
Result	Pass / Fail

Test Case 4: UPSERT Persistence — GitHub & LinkedIn JSONB Storage

<i>Test case</i>	UPSERT Persistence — GitHub & LinkedIn JSONB Storage
Related Feature	GitHubIngestionService / LinkedInIngestionService — Supabase UPSERT on github_profiles and linkedin_profiles tables
Context	Scraped profile data must be stored in PostgreSQL via UPSERT. The first run inserts a new record; a second run with updated data must overwrite the existing record — not create a duplicate. Nested JSONB fields (language maps, experience arrays with structured date objects) must be preserved accurately.
Input Data	First run: candidate_uuid = "uuid-tc4", GitHub data with 15 repos, top_languages = {"Python": 72.3, "Shell": 27.7}, LinkedIn experience with start_date = {year: 2021, month: "Jan"}. Second run (same UUID): updated GitHub data with 22 repos, top_languages = {"Python": 55.0, "TypeScript": 45.0}
Expected Output	After first run: one record each in github_profiles and linkedin_profiles keyed by candidate_uuid. After second run: row count for candidate_uuid remains 1 in each table; public_repos_count updated to 22; top_languages reflects the new distribution. LinkedIn experiences JSONB retains nested date structure as {year: 2021, month: "Jan"} (not a flat string).
Test Steps	1. Call the GitHub ingestion service with the first run data. Assert 1 row inserted and public_repos_count = 15. 2. Call the same service again with updated data (22 repos).

	Assert row count is still 1 and public_repos_count = 22. 3. Call the LinkedIn ingestion service. Query the stored record and assert the experiences[0].start_date is a structured object {year: 2021, month: "Jan"}.
Actual Output	GitHub: 1 row in github_profiles, updated to public_repos_count = 22 with no duplicates. LinkedIn: experiences[0].start_date = {year: 2021, month: "Jan"} stored correctly as nested JSON.
Result	Pass / Fail

Test Case 5: Radar Analytics — Skill Score & Confidence Calculation

<i>Test case</i>	Radar Analytics — Skill Score & Confidence Calculation
Test Case	Radar Analytics — Skill Score & Confidence Calculation
Related Feature	generate_analytics + analyze_github_local_fallback — Radar Skill Matrix & Confidence Score
Context	The analytics engine must deterministically compute radar skill scores from GitHub repository metadata, apply a confidence score formula, scale pre-enrichment scores proportionally, and enforce an upper bound on the confidence output.
Input Data	GitHub profile with repos named "fastapi-backend" and "docker-deploy", top_languages = {"Python": 71.4, "Dockerfile": 28.6}, readme_content = "fastapi postgresql docker kubernetes". Confidence test input: all 5 skill dimensions at 100.0 with 8 semantic tags.
Expected Output	Skill scores: Backend = 86 (base 25 + 3 keyword hits × 12 + language bonus), Cloud Dev = 71. Pre-enrichment scores = post-enrichment × 0.75 (e.g., Backend pre = 64.5). Confidence score when all dimensions are 100 and 8 tags: raw = $100 \times 0.8 + 8 \times 5 = 120 \rightarrow$ capped at 99.0. Default fallback tags ["#github", "#enriched"] returned when no keywords are matched.
Test Steps	1. Build the GitHub profile with the inputs above. 2. Run analyze_github_local_fallback. Assert Backend = 86 and Cloud Dev = 71.

	3. Assert pre_enrichment scores equal post_enrichment $\times 0.75$ for each dimension. 4. Run generate_analytics with all-100 skills and 8 tags. Assert match_confidence_score = 99.0. 5. Run generate_analytics with no input data. Assert semantic tags fall back to ["#github", "#enriched"].
Actual Output	Backend = 86, Cloud Dev = 71, pre_enrichment scaled correctly at $0.75\times$. Confidence capped at 99.0. Default tags returned when no keywords found.
Test Steps	Pass / Fail

Test Case 6: Enrichment Worker — End-to-End Pipeline

<i>Test case</i>	Enrichment Worker — End-to-End Pipeline
Related Feature	enrichment_worker — full 8-step enrichment pipeline
Context	A fully configured candidate with both a GitHub username and a LinkedIn URL triggers the enrichment workflow. All external services (Supabase, GitHub API, Apify) are available and return valid responses. The pipeline must complete all steps in sequence and notify the client via WebSocket.
Input Data	candidate_uuid = "uuid-e2e-001", github_username = "testdev", linkedin_url = "https://www.linkedin.com/in/testdev/", all service mocks return valid data, one active WebSocket subscriber registered for this candidate
Expected Output	1. Candidate record created/confirmed in Supabase. 2. Social links updated in the candidates table. 3. GitHub profile successfully scraped and stored. 4. LinkedIn profile successfully scraped and stored. 5. Radar analytics generated with at least one non-zero skill dimension. 6. In-memory enrichment status updated to ENRICHED. 7. WebSocket subscriber receives a JSON frame with status = "ENRICHED", containing data, skills_matrix, career_trajectory, and semantic_tags fields.
Test Steps	1. Seed the candidate store with the candidate above. 2. Register a mock WebSocket for candidate_uuid.

	3. Execute the enrichment worker. 4. Assert status transitions: IN_PROGRESS at start → ENRICHED at completion. 5. Assert both github_profiles and linkedin_profiles UPSERT calls were made. 6. Assert the WebSocket received exactly one frame with status = "ENRICHED" and non-empty skills_matrix.
Actual Output	All 6 pipeline steps completed. Final status: ENRICHED. WebSocket frame delivered: {"status": "ENRICHED", "skills_matrix": {...}, "career_trajectory": [...], "semantic_tags": [...]}.
Result	Pass / Fail

3.2.6 ABAC Security & Scheduling

Written by: Nguyễn Tiến Cường (24127337)

Edited by: Nguyễn Khánh Toàn (24127252)

Reviewed by: Hồ Đình Trí (24127254)

Test Case 1: ABAC Full Access Clearance

Test Case	Test Case 1: ABAC Full Access Clearance
Related Feature	ABAC Security Engine — apply_abac (HR Clearance)
Context	Authenticated HR Manager queries candidate details for application screening and background verification.
Input Data	Candidate profile payload with full_name: "Nguyen Van B", email: "vanb.nguyen@example.com", phone: "+84912345678", salary_expectation: "3500 USD", role: "hr".
Expected Output	100% unmasked payload. All PII fields (full_name, email, phone, salary_expectation, linkedin.avatar_url) returned verbatim.
Test Steps	1. Load candidate enriched profile. 2. Invoke apply_abac(profile, role="hr").

	3. Assert all PII fields match the original source values. 4. Verify source object immutability.
Actual Output	Full unmasked JSON profile returned. All PII and technical fields intact.
Result	Passed

Test Case 2: ABAC Tech Role PII Masking

Test Case	ABAC Tech Role PII Masking
Related Feature	ABAC Security Engine — apply_abac (Tech Lead PII Redaction)
Context	Technical Lead reviews candidate profile to evaluate technical competency while preventing cognitive and identity bias.
Input Data	Complete candidate profile payload, User role: "tech_lead".
Expected Output	PII fields redacted to "****". Technical fields (technical_skill_matrix, top_languages, semantic_tags, public_repos_count, headline, company) preserved intact.
Test Steps	1. Provide enriched profile payload. 2. Invoke apply_abac(profile, role="tech_lead"). 3. Assert full_name, email, phone, address, salary_expectation, linkedin.avatar_url are "****". 4. Assert top_languages and analytics match source data.
Actual Output	PII fields redacted to "****". Technical metrics (Go: 24.5%, Python: 65.5%, Confidence: 94.0) visible.
Result	Passed

Test Case 3: ABAC Default-Deny & Type Preservation

Test Case	ABAC Default-Deny & Type
-----------	--------------------------

	Preservation
Related Feature	ABAC Security Engine — Dynamic Policy & _mask_value
Context	System schema introduces new unwhitelisted data fields with diverse primitive and composite data types.
Input Data	Payload with unwhitelisted fields: date_of_birth: "1995-12-25", internal_credit_score: 780.5, is_blacklisted: False, private_tags: ["vip"].
Expected Output	Default-deny applies: date_of_birth becomes "****", internal_credit_score becomes 0, is_blacklisted becomes False, private_tags becomes []. No Pydantic validation error.
Test Steps	1. Construct payload with novel unwhitelisted fields of varying data types. 2. Call apply_abac(payload, role="tech_lead"). 3. Assert string values masked to "****", numeric to 0, boolean to False, list to [].
Actual Output	Unwhitelisted fields masked according to type preserving rules. Zero schema serialization exceptions.
Result	Passed

Test Case 4: Sweep-Line Multi-Interviewer Overlap

Test Case	Sweep-Line Multi-Interviewer Overlap
Related Feature	SweepLineService — find_slots
Context	Recruiter schedules a panel interview involving multiple interviewers with partially overlapping working hours.
Input Data	Interviewer 1 free (08:00 - 10:30 UTC), Interviewer 2 free (08:00 - 10:00 UTC), min_slot_minutes = 45.
Expected Output	Overlap window 08:00 - 10:00 UTC partitioned into consecutive 45-minute slots (08:00-08:45,

	08:45-09:30). Each slot contains all interviewer IDs.
Test Steps	1. Construct interviewer_freebusy mapping. 2. Execute SweepLineService.find_slots. 3. Assert returned slot count >= 2. 4. Assert slot start/end times and 45.0m duration.
Actual Output	2 valid slots generated (08:00-08:45, 08:45-09:30). All interviewer IDs mapped.
Result	Passed

Test Case 5: Sweep-Line Sub-Minimum Discard (< 45m)

Test Case	Sweep-Line Sub-Minimum Discard (< 45m)
Related Feature	SweepLineService — Boundary Gating
Context	Interviewers have a brief common free interval that is insufficient for a complete interview session.
Input Data	Interviewer 1 free (08:30 - 09:30 UTC), Interviewer 2 free (09:00 - 10:30 UTC), common overlap = 30 min (09:00 - 09:30 UTC), min_slot_minutes = 45.
Expected Output	Empty list [] returned. 30-minute interval discarded as it fails the 45-minute minimum threshold.
Test Steps	1. Populate freebusy map with 30m overlap. 2. Run find_slots(min_slot_minutes=45). 3. Assert result is [].
Actual Output	Result list empty []. Sub-45m interval successfully discarded.
Result	Passed

Test Case 6: Sweep-Line Disjoint Schedules (Zero Overlap)

Test Case	Sweep-Line Disjoint Schedules (Zero Overlap)
Related Feature	SweepLineService — Conflict Resilience
Context	Selected interviewers have completely disjoint, non-overlapping availability windows.
Input Data	Interviewer 1 free (08:00 - 09:00 UTC), Interviewer 2 free (09:00 - 10:00 UTC).
Expected Output	Empty list [] returned. Algorithm logs scheduling.sweepline.complete with slots_found=0. No exceptions raised.
Test Steps	1. Setup disjoint schedule intervals. 2. Call SweepLineService.find_slots. 3. Assert return value is []. 4. Verify no infinite loop or crash.
Actual Output	[] returned within 2ms. Log event emitted.
Result	Passed

Test Case 7: Google Calendar Token Refresh Resilience

Test Case	Google Calendar Token Refresh Resilience
Related Feature	SchedulingService & GoogleOAuthService — Token Auto-Refresh
Context	Querying interviewer FreeBusy calendar when the cached Google Calendar OAuth access token has expired.
Input Data	Interviewer with expired calendar_api_key and valid calendar_refresh_token = "valid_refresh_token_xyz".
Expected Output	System detects empty response, invokes GoogleOAuthService.refresh_access_token, updates DB repository with new key, and successfully queries FreeBusy intervals.
Test Steps	1. Mock GoogleCalendarService to return [] on

	first attempt and valid intervals on retry. 2. Mock GoogleOAuthService.refresh_access_token. 3. Execute SchedulingService.query_slots. 4. Assert refresh service invoked and new key saved.
Actual Output	Access token refreshed seamlessly. FreeBusy intervals retrieved and slots computed.
Result	Passed

Test Case 8: Interview Booking & Multi-Channel Notification

Test Case	Interview Booking & Multi-Channel Notification
Related Feature	SchedulingService — confirm_slot & Multi-Channel Notifiers
Context	HR confirms selected interview time slot for candidate and interviewer panel.
Input Data	Selected UTC slot (02:00 - 02:45 UTC / 09:00 - 09:45 GMT+7), candidate email, interviewer IDs.
Expected Output	ConfirmedSlot entity created in DB, Google Calendar event created with Meet link, email notification dispatched to candidate and interviewers, Slack notification dispatched with GMT+7 formatted time.
Test Steps	1. Dispatch confirm_slot request. 2. Assert ConfirmedSlot persisted. 3. Verify CalendarEventService.create_event called. 4. Verify EmailNotifier and SlackNotifier dispatched.
Actual Output	Booking confirmed with Google Meet URL. Slack and Email notifications dispatched in GMT+7 format.
Result	Passed

4

AI Usage Declaration

In alignment with academic integrity guidelines and responsible engineering principles, our team utilized Generative AI assistance (specifically Google Gemini) strictly as an augmented productivity tool throughout the Software Testing phase of the SmartATS project.

The boundaries and specific applications of AI usage within this phase are defined as follows:

- **Test Plan & Document Formatting:** AI was leveraged to assist in structuring initial test report layouts, refining English technical phrasing, and formatting test case specification tables to ensure compliance with the standardized testing template.
- **Test Case Boilerplate & Boundary Ideation:** Generative AI served as an ideation partner to suggest boundary value analysis edge cases (such as strict file size thresholds, sub-minimum schedule overlaps, and empty scraping datasets) and to scaffold repetitive unit/integration test fixtures using pytest and pytest-asyncio.
- **Mock Schema & Data Scaffolding:** AI was used to draft sample mock JSON payloads for third-party services (such as GitHub REST API responses, Apify LinkedIn scrapers, and Google Calendar FreeBusy intervals).

Human Oversight & Accountability Statement:

- **Zero Automated Test Validation:** All core verification logic—including the dynamic ABAC data masking rules, pgvector semantic search assertions, idempotent data merge conflict resolution, and the Sweep-Line interval overlap checks—were designed, implemented, and verified strictly by team members. Every automated test script and mock assertion was executed and validated against actual backend endpoints and databases. The team assumes 100% full accountability for all test suites, reported metrics, and verification outcomes presented in this document.

5

Presentation

Link video Youtube: <https://youtu.be/oyjEdv-HaAA>

6

Reflective Report

Which sections of this template are most helpful for implementation, and which are unnecessary? Provide specific examples to support your reasoning.

Most Helpful Sections for Implementation

- **Section 3.2 (Test Case Specifications):** This is the most valuable section for engineering execution because it directly translates architectural requirements into executable unit and integration tests. For instance, specifying precise input/output pairs—such as validating %PDF-Magic Bytes and enforcing the $\leq 10\text{MB}$ payload boundary in Flow A, or asserting that non-string data types (0, False, []) are preserved during ABAC dynamic PII masking in Flow D—provided explicit guidelines for writing automated pytest test suites without ambiguity.
- **Section 2.2 (Testing Approaches & Techniques):** This section establishes necessary architectural boundaries for testing microservice dependencies. Formulating rules for Boundary Value Analysis (BVA) on Sweep-Line calendar scheduling (e.g., discarding sub-45-minute overlapping windows) and defining external dependency virtualization (mocking the Apify LinkedIn scraper and Google Calendar v3 FreeBusy API) ensured that automated test pipelines remained deterministic, fast, and free from external API costs.

Unnecessary or Redundant Sections

- **Section 3.1 (List of Test Cases):** Section 3.1 functions as a high-level summary that entirely duplicates the sequence, feature names, and brief descriptions detailed in Section 3.2. Maintaining both lists across all four backend flows (Ingestion, AI Parsing, Scrapers, and ABAC/Scheduling) creates documentation overhead and synchronization drift when test titles or scopes change during development.
- **Section 5 (Presentation Link Field):** Embedding video submission URLs directly within a software testing document mixes course administrative deliverables with functional software artifacts. Video submission fields are better handled in academic portals or project README files rather than embedded inside a technical testing specification.